

VaakSetu (वाक् सेतु) — Complete Project Explainer

The Voice Bridge for document safety in rural India

Repository	https://github.com/abhi-yanta/VaakSetu
Audience for this doc	Founder / teammate with zero prior context
Primary product today	Flutter Android app in <code>mobile/</code>
Last verified against	Repo code as of overview generation

Table of contents

1. [What it is \(in one minute\)](#)
 2. [Why it exists](#)
 3. [Who it is for](#)
 4. [Problems it solves](#)
 5. [Design principles](#)
 6. [User journey \(app flow\)](#)
 7. [Major features](#)
 8. [Key screens and UI](#)
 9. [Tech stack](#)
 10. [How speech works \(TTS\)](#)
 11. [OCR and analysis pipeline](#)
 12. [Form Field Guide](#)
 13. [Datasets and presets](#)
 14. [Project structure](#)
 15. [How to run the app](#)
 16. [What has been built so far](#)
 17. [Limitations and gaps](#)
 18. [Related folders \(web, mockups\)](#)
-

1. What it is (in one minute)

VaakSetu means *Voice Bridge* (वाक् = voice/speech, सेतु = bridge).

It is an **audio-first Android app** that helps people who struggle to read protect themselves when dealing with:

- Loan papers and microfinance agreements
- Land / sale deeds and stamp papers
- Labor / job contracts
- Hospital consent / liability forms
- Government-style **registration forms** (name, phone, KYC fields, etc.)

Core idea: Point a low-cost phone camera at a document → the app reads the text on-device → speaks a clear safety summary (or walks through form fields) in the user's language.

The flagship experience lives in `mobile/` (Flutter). There is also an older **React web/kiosk** path under `client/` + `server/`, and a `dataset/` hub for legal/OCR reference data.

2. Why it exists

In rural and semi-urban India, many people are functionally low-literacy. Complex English or legalese on stamp paper is effectively invisible to them. Common harms include:

1. **Predatory loans** — high interest (often framed as “per month”), blank cheques, land as collateral for small cash.
2. **Land grabs** — papers presented as “loan receipts” that are actually sale deeds or irrevocable powers of attorney.
3. **Exploitative labor** — bonded-labor style clauses, unpaid overtime, resignation penalties.
4. **Medical waivers** — emergency signatures that wipe hospital liability.
5. **Form anxiety** — even when a form is “safe,” people cannot tell what each blank means.

Cloud-only AI fails in villages with weak connectivity and raises privacy concerns for land and ID documents. VaakSetu's intended answer: **on-device OCR + explainable rules + spoken guidance**.

3. Who it is for

Group	Need
Rural / low-literacy adults	Hear warnings without reading
Semi-literate workers & farmers	Spot red-flag clauses before signing
Family helpers	Guide a relative through a form field-by-field
CSC / legal-aid volunteers (web/kiosk vision)	Assist others on a shared laptop

Languages in the mobile app (12): Hindi, Tamil, Telugu, Marathi, Bengali, Gujarati, Kannada, Malayalam, Odia, Punjabi, Assamese, Urdu.

UI strings, spoken prompts, and form field help are localized via `LocalizedContent` (plus English labels in the form dictionary).

4. Problems it solves

Problem	How VaakSetu responds
"I can't read this paper"	Speaks category + severity + warning lines aloud
"Is this a dangerous loan?"	Rule engine scores interest / collateral / blank-sign patterns
"Is this even a legal document?"	Marks non-legal text as unrecognized (not "safe to sign")
"Photo is blurry"	Marks unclear; asks to rescan
"What do I write in this form blank?"	Form Field Guide: match OCR labels → spoken help per field
"Say that again"	फिर से सुनो (Listen again) on guided screens + AppBar speaker

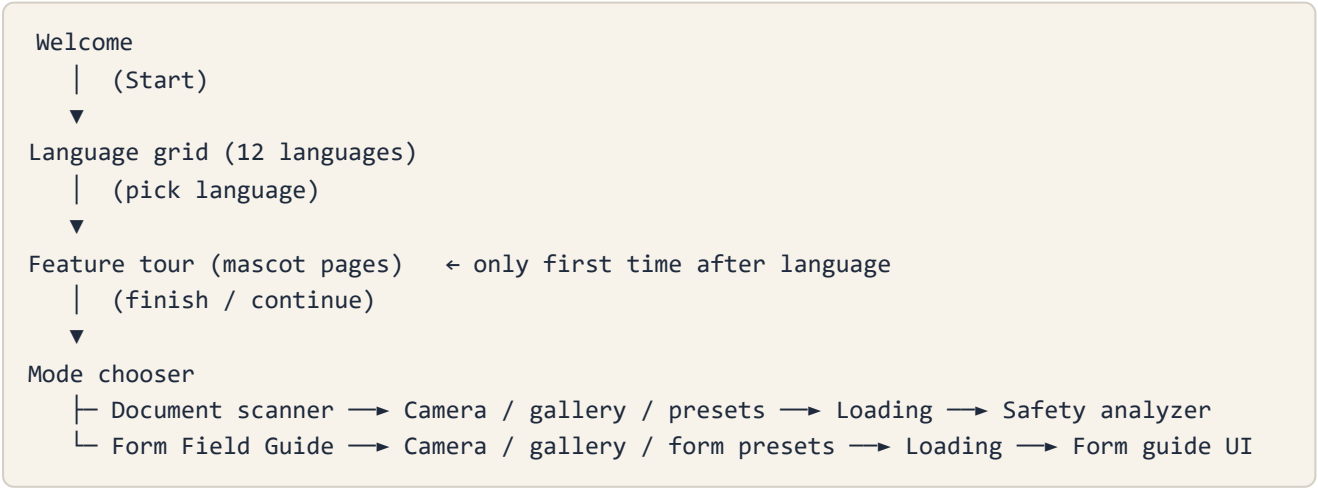
5. Design principles

From product docs and code behavior:

1. **Offline-first for core safety** — ML Kit OCR and the Dart rule engine run on device. Documents need not leave the phone for analysis. Online TTS is optional enhancement.
2. **Audio + color + haptics** — green / yellow / red / blue severity; vibration intensity rises with danger.
3. **Don't cry wolf / don't false-reassure** — ambiguous notebooks must not get a green "safe" badge.
4. **Large touch targets** — tactile buttons, simple grids, cream light theme for outdoor readability.

6. User journey (app flow)

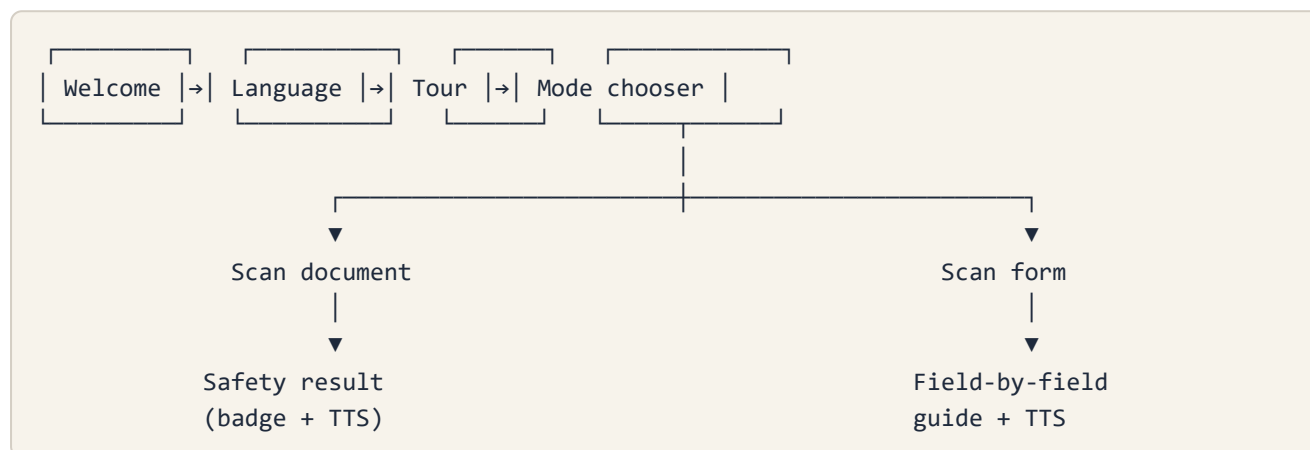
Exact navigation is owned by `mobile/lib/main.dart` (`AppView` enum).



Notes verified in code:

- After the tour once (`_tourCompleted`), choosing language again skips the tour and goes to the mode chooser.
- Form mode can also open if document mode OCR text "looks like a form" (`FormFieldEngine.looksLikeForm`).
- AppBar logo tap returns to language selection.
- Back from scanner returns to mode chooser.
- TTS pauses when the app goes to background and can resume when returning.

Simple flow diagram



7. Major features

7.1 Legal document safety analyzer

User view: After a photo or preset, the app announces what kind of paper it thinks it is and whether it looks safe, cautionary, or dangerous. Warnings can be tapped to hear again. Raw OCR text can be heard word-by-word or sentence-by-sentence.

Tech view:

- `OcrService` → text
- `RuleEngine.analyze` → `DocumentAnalysis` (category, severity, warning keys, raw text, confidence)
- `DocumentAnalyzerView` → `SecurityBadge`, wave visualizer, localized warnings, interactive readers

Categories: loan, deed, job, medical, unrecognized, unclear.

Severities: safe, warning, danger, unknown.

Example danger patterns (rule-based, not an LLM): interest above ~24% APR, land collateral on small loans, bonded-labor style clauses, blanket medical liability waivers.

7.2 Form Field Guide

User view: Choose “form” mode, scan an application/KYC-style sheet (or pick a demo form). The app steps through fields (name, DOB, phone, address, ...), speaks what each blank means, and shows where to write (below / right heuristics).

Tech view: `FormFieldEngine` dictionary matching + `FormFieldGuideView` interactive canvas. Runtime dictionary is Dart (`FormFieldDictionary`); JSON under `dataset/form_guide/` is an export for docs/ML, not the live loader.

7.3 Camera scanner + presets

- Live camera preview, shutter, gallery picker, flash (where supported).
- Framing prompt spoken in the selected language.
- **Offline demo presets** for loans, deeds, labor, medical, and five form samples (scholarship, ration card, KYC, etc.) so demos work without a printed page.

7.4 Multilingual voice UI

- Welcome / language / tour / mode / scan / processing / result prompts.
- Per-warning narration.
- Listen-again on most guided screens.
- AppBar mute / replay control synced to `TtsService` (`ChangeNotifier`).

7.5 Accessibility extras

- Haptic patterns (`HapticService`) — light tap, warning, danger.
- Pulsing / orbiting logo animation while analyzing.
- Cream light surfaces (`AppColors`) for rural outdoor use.

7.6 What is *not* built (speech input)

There is **no speech-to-text (STT)** / voice-command pipeline in the Flutter app today. Interaction is tap + TTS playback. (Camera audio is disabled.)

8. Key screens and UI

Screen	File (approx.)	What the user sees
Welcome	welcome_view.dart	Logo, वाक् सेतु title, Hindi start prompt, Start button
Language	language_selector_view.dart	12-language grid with script badges; listen-again
Feature tour	character_welcome_view.dart	Mascot poses + 4 pages: scan, safety, forms, languages
Mode chooser	mode_chooser_view.dart	Document scanner vs Form Field Guide
Scanner	camera_scanner_view.dart	Viewfinder, shutter, gallery, preset list
Loading	main.dart	Animated logo + “analyzing” copy
Analyzer	document_analyzer_view.dart	Color badge, listen-all, warnings, text readers
Form guide	form_field_guide_view.dart	Field list / canvas walkthrough

Visual language

- **Background cream:** #F7F3EB (AppColors.bgDark — name is historical; theme is light).
- **Accents:** saffron / deep teal.
- **Safety:** green / amber / red / cyan info.
- **Mascot:** guide character with poses (point, wave, namaste, celebrate).
- **फिर से सुनो:** ListenAgainButton — localized “listen again” control.

UX mockups note

docs/ux-mockups/ contains exploratory images (A–D listen / helper concepts). Those were **rejected as product direction** and are **not** the shipped UI. The live product is the cream Flutter flow above.

9. Tech stack

Mobile (primary)

Layer	Choice
Framework	Flutter 3.x, Dart
OCR	<code>google_mlkit_text_recognition</code> (on-device)
Camera / gallery	<code>camera</code> , <code>image_picker</code>
TTS playback	<code>flutter_tts</code> , <code>audioplayers</code>
Networking (optional TTS)	<code>http</code>
State / prefs	<code>provider</code> pattern via <code>ChangeNotifier</code> , <code>shared_preferences</code>
Haptics	<code>vibration</code>

See `mobile/pubspec.yaml` for pinned versions.

Web & kiosk (secondary)

- `client/` — React + Vite PWA-style document UI (Tesseract / browser TTS historically described in root README).
- `server/` — Express hotspot/kiosk helper.

Treat the **Flutter app** as the current source of truth for features; the root README still emphasizes the older web stack.

Data / tooling

- `dataset/` — legal sources + synthetic 88-case benchmark + form-guide export.
- `mobile/tool/` — `run_with_env.ps1` , dataset generate/evaluate, mascot tooling, form export tests.

10. How speech works (TTS)

Implemented in `mobile/lib/data/services/tts_service.dart` .

Priority cascade:

Layer 1	Bhashini / ULCA (MeitY)	← needs BHASHINI_* keys
	↓ if missing / fails	
Layer 2	HuggingFace AI4Bharat (indic-parler-tts)	← needs HF_TOKEN
	↓ if missing / fails	
Layer 3	Device neural TTS (flutter_tts, slow rate ~0.45)	← always available offline

Important:

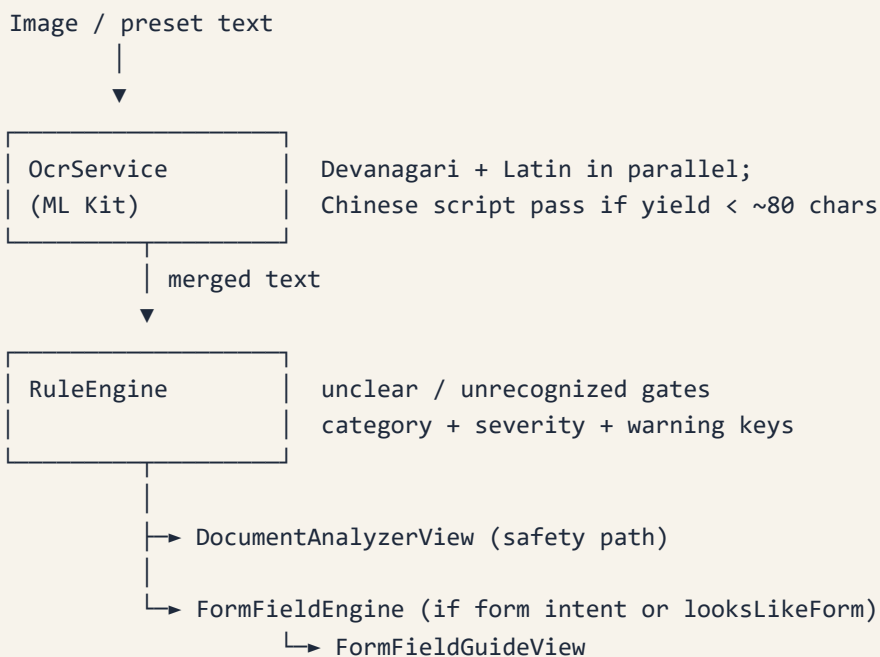
- Keys are injected at run time via `--dart-define` (never hard-coded).
- Use `mobile/.env.example` as a template; copy to `.env` locally.
- **Never commit real keys.** This overview intentionally shows empty placeholders only.

Conceptual `.env` keys:

```
BHASHINI_USER_ID=
BASHINI_UDYAT_KEY=
BASHINI_INFERENCE_KEY=
HF_TOKEN=
```

Without any online keys, the app still speaks using **device TTS**.

11. OCR and analysis pipeline



OCR details live in `ocr_service.dart`. Rule details live in `rule_engine.dart`. Repository wiring: `document_repository.dart`.

12. Form Field Guide

Item	Detail
Field keys	~22 (name, dob, phone, address, id_number, bank_account, ifsc, ...)
Matching	Keyword / similarity / token overlap against OCR lines
Languages for field help	13 codes including English
Demo forms	kisan welfare, scholarship, ration card, bank KYC, job application
Blank placement	Heuristics: below vs right of label

This is **dictionary + heuristics**, not a neural layout model. Layout-aware DocAI is future work (notes exist in `dataset/form_guide/README.md`).

13. Datasets and presets

`dataset/sources/`

1. Indian Kanoon samples / notes
2. State registry / deed checklists
3. RBI fair practices rules
4. CUAD (contract understanding) references
5. CLAUDETTE unfair-clause patterns
6. AI4Bharat Indic OCR catalogs

`dataset/synthetic/`

- ~88 labelled samples + evaluate script for rule-engine benchmarking.

App assets

- `mobile/assets/presets/` — scanner demos.
 - `mobile/assets/dataset/form_guide/` — exported JSON mirror.
 - `mobile/assets/mascot/` , `mobile/assets/icon/` .
-

14. Project structure

```
vaaksetu/
├─ mobile/                ← Flagship Flutter Android app
│  └─ lib/
│     ├── main.dart       ← App shell & navigation
│     ├── data/           ← OCR, TTS, haptics, presets, repository
│     ├── domain/         ← Models, LocalizedContent, RuleEngine, FormFieldEngine
│     └─ ui/              ← Screens + shared widgets
│  └─ assets/
│  └─ tool/run_with_env.ps1
│  └─ .env.example
│  └─ test/
│  └─ pubspec.yaml
├─ dataset/              ← Legal / OCR / form datasets
├─ client/               ← React web UI (kiosk / CSC path)
├─ server/               ← Express companion for web
├─ docs/                 ← This explainer + ux-mockups (exploratory only)
├─ README.md             ← Older web-first overview
└─ VAAKSETU_SYSTEM_DESCRIPTION.md ← Deep system/domain notes for developers
```

15. How to run the app

Prerequisites

- Flutter SDK (project uses Flutter 3.x)
- Android Studio / SDK (API 21+)
- Android phone or emulator with camera (optional for presets)

Install & test

```
cd mobile
flutter pub get
flutter test
flutter run
```

Optional online TTS

```
cd mobile
Copy-Item .env.example .env
# Edit .env locally with your own keys (do not commit)
.\tool\run_with_env.ps1
```

`run_with_env.ps1` reads `.env` / `.env.local` and passes non-empty values as `flutter run --dart-define=KEY=value` without printing secrets.

Release APK

```
cd mobile
flutter build apk --release
# Output: mobile/build/app/outputs/flutter-apk/app-release.apk
```

Web stack (optional)

```
cd server && npm install && npm start # often :5000
cd client && npm install && npm run dev # often :3000
```

16. What has been built so far

Narrative of capabilities from recent commit themes (newest first):

1. **Rural UX polish** — cream light theme, listen-again replay, mascot alpha fixes.
2. **Form Field Guide** — interactive canvas, TTS walkthrough, form presets, mode chooser.
3. **Mascot feature tour** after language selection.
4. **Startup motion** — pulsing logo; orbiting ring on long OCR loads.
5. **TTS lifecycle** — pause on background; smart resume on foreground.
6. **Document verbalizer** — full-document + sentence-level audio on OCR text.
7. **Tap-to-speak words** on raw text.
8. **Full localization** of analyzer UI / loading / speech across 12 Indic languages.
9. **Language cards** use script-character badges (not flag emoji).
10. **Camera layout fixes** — correct aspect ratio, shutter always reachable.
11. **Stamp-paper OCR resilience** + AI4Bharat/Bhashini TTS layers + dataset hub.
12. **Dataset folder** with six sources + synthetic 88-case benchmark.
13. **Initial Flutter app** + classification rules + ML Kit OCR.

Older history also includes the React client, Express server, and early camera work.

17. Limitations and gaps

Be honest with yourself when demoing:

Area	Current state
STT / voice commands	Not implemented
True offline “AI voice”	Device TTS only unless Bhashini/HF configured
Legal analysis	Deterministic keyword/rules — not a lawyer, not an LLM
OCR quality	Struggles on bad lighting, tiny stamp fonts, some scripts
Form guide layout	Heuristic blank positions; not trained pixel layout detection
Form dictionary runtime	Dart source of truth; JSON export is secondary
iOS	Project is Android-oriented; iOS packaging not the focus
Web vs mobile parity	Features diverge; Flutter is ahead for rural UX
Root README	Still describes PWA/Tesseract-first story; prefer this doc + <code>VAAKSETU_SYSTEM_DESCRIPTION.md</code> for mobile truth
UX mockups A–D	Exploratory only — not product

18. Related folders (web, mockups)

- `client/` + `server/` — institutional kiosk / browser path; useful for CSC demos, not the cream mobile UX.
 - `docs/ux-mockups/` — discarded direction sketches.
 - `VAAKSETU_SYSTEM_DESCRIPTION.md` — longer developer/domain reference (some TTS layer naming may lag the code; **code order is Bhashini → HuggingFace → device**).
-

Quick “forgotten founder” cheat sheet

Question	Answer
What did we build?	Audio-first Flutter app that scans docs/forms and speaks risks or field help
For whom?	Low-literacy / rural Indian users needing spoken safety
Killer modes?	(1) Document danger check (2) Form field walkthrough
Offline?	OCR + rules yes; best voices need optional internet keys
How to demo without papers?	Use built-in presets on the scanner screen
How to run with nice voices?	<code>.env</code> + <code>.\tool\run_with_env.ps1</code> (never commit secrets)
Where is truth?	<code>mobile/lib/</code> code > this overview > older root README

Generated as an internal explainer. No API keys or secrets included.